

AGENT-READY PRODUCT SPECIFICATION

DocRot Detector

Documentation Rot Detector

Version	1.0
Build Plan	3 Phases over 10 weeks
Stack	React 19 · FastAPI · Node · PostgreSQL · AWS + Docker
Auth	
Last Updated	April 3, 2026

Read this entire specification before writing a single line of code.

This document is the single source of truth for all AI coding agents. When the spec and your judgment conflict, implement the spec and add a `// SPEC QUESTION:` comment.

Table of Contents

1. Product Overview
2. Tech Stack
3. Project File Structure
4. Data Models
5. Phase 1 — Planning & Design
6. Phase 2 — Core Scanner MVP
7. Phase 3 — Backend, Database & Web Dashboard
8. API Reference (Full)
9. CLI & Output Guidelines
10. Error Handling Standards
11. Non-Functional Requirements
12. Environment Variables
13. Glossary
14. Agent Instructions & Constraints

1. Product Overview

Documentation Rot Detector is a developer tool that automatically detects when project documentation becomes outdated as source code evolves. It integrates directly into development workflows and continuously scans repositories to identify mismatches between code and documentation. DocRot Detector solves this by analyzing both source code and documentation using static analysis and Abstract Syntax Tree (AST) parsing. The system extracts structured information from code, assigns hashes to code elements, and links those elements to their references in documentation. When code changes in a way that affects documentation, the system automatically flags the affected sections and generates a clear report.

The tool can run locally through a command line interface or automatically through Git hooks and CI pipelines. In later phases, it can optionally use lightweight AI models to validate mismatches and suggest updated documentation text.

Current Implementation Status (March 27, 2026)

The project is currently in Phase 3 (Backend, Database, and Dashboard integration).

Recent updates include:

- Migration from SQLite-based storage to PostgreSQL hosted via Supabase
- Initial frontend-backend integration with live dashboard data
- Authentication system in progress (GitHub OAuth and JWT-based protection)
- Ongoing research into deployment strategy and hosting options

1.1 Target Users

- Software development teams maintaining large or fast-moving repositories
- Open-source maintainers managing community documentation
- Technical writers responsible for API or developer documentation
- Engineering organizations onboarding new developers to complex systems

1.2 Core Value Proposition

- Automated detection of outdated documentation — identifies mismatches between code and documentation automatically
- AST-based code analysis — accurately extracts functions, classes, parameters, and other code elements
- Clear mismatch reporting — provides readable reports showing exactly which documentation sections need review
- CI/CD integration — runs automatically on commits or pull requests using Git hooks or GitHub Actions
- AI-assisted suggestions (Phase 3) — proposes updated documentation wording or examples when mismatches are detected

Platform	CLI scanner + Web dashboard (React). Integrates with development workflows and CI pipelines.
----------	--

Auth	Admin login or token-based authentication (MVP). JWT sessions for dashboard. Optional GitHub OAuth / GitHub App integration.
Persistence	PostgreSQL database (hosted via Supabase). Stores projects, scan history, code elements, documentation references, and detected issues. The system has transitioned from earlier SQLite-based experimentation to PostgreSQL due to improved support for concurrent access and real-time updates across backend and frontend services.
Offline	CLI scanner runs locally without internet. Dashboard requires backend API.
Environments	Development (local CLI + backend) · Staging (Docker deployment) · Production (hosted backend + dashboard) · CI/CD via GitHub Actions.

Updated Implementation (April 2026)

The system architecture has evolved from the original specification to support real-time data synchronization, simplified deployment, and improved authentication.

Key updates include:

- Database Migration:
The system has transitioned from PostgreSQL (Supabase) to Firebase Firestore, enabling real-time updates and easier frontend integration.
- Authentication:
Authentication is now handled through Firebase Authentication with GitHub OAuth, replacing the earlier JWT-only approach. Users authenticate via GitHub and are managed within Firebase Auth.
- API & Data Flow Changes:
Instead of a traditional FastAPI-only pipeline, scan results are now ingested through a Google Cloud Function endpoint (*ingestScan*), which writes directly to Firestore.
- Frontend Integration:
The React dashboard now pulls live data directly from Firestore using Firebase SDK integration, replacing earlier REST-based API calls for core dashboard data.
- Hosting & Deployment:
The frontend is deployed using Firebase Hosting, and backend ingestion is handled via Google Cloud Functions, replacing the originally planned AWS + Docker deployment for the MVP.
- CI/CD Integration:
GitHub Actions now authenticate with Google Cloud using Workload Identity Federation (OIDC) and send scan results securely to the backend ingestion endpoint.

2. Tech Stack

All technology decisions below are fixed. Do not substitute libraries without adding a // ARCH DECISION: comment explaining the reason.

Layer	Technology	Notes / Constraints
-------	------------	---------------------

Frontend	React 19	Web dashboard: login, project select, scan history, issues list/detail, search/filter.
Styling	Figma (design) + TailwindCSS	Keep UI clean and consistent with Figma; prioritize readability for issue/report views.
State	React state + lightweight client state (no hard requirement)	Store UI state (selected project/scan, filters, search query); treat scans/issues as server data.
Backend	FastAPI	Serves dashboard APIs and connects to DB; exposes scan + issue endpoints.
Database	Firebase Firestore (replacing PostgreSQL for real-time data storage and simplified integration)	Persists projects, scans, code elements, doc refs, issues; ensure foreign keys by project_id/scan_id.
Auth	Firebase Authentication with GitHub OAuth integration (JWT handled internally by Firebase)	MVP can be minimal; JWT only if needed for sessions; keep secrets server-side.
Validation	FastAPI + Pydantic models	Validate all request bodies and return consistent response schemas for the frontend/CLI.
Date/Time	Python datetime (ISO 8601)	Store timestamps for scans/issues; return ISO strings to frontend.
Charts	React chart library (optional for dashboard)	Only needed if you implement analytics/historical tracking; skip for MVP if time.
HTTP Client	Fetch API or Axios	Centralize API calls from the dashboard; include auth token on requests.
AI	OpenAI	AI optional; used only for validation + suggestion generation after deterministic mismatch detection.
Testing	Pytest (backend/scanner) + frontend tests as needed	Prioritize unit tests for AST extraction, hashing, and mismatch rules; add API tests for core endpoints.
Hosting	Firebase Hosting (frontend) + Google Cloud Functions (backend ingestion)	GitHub Actions runs scans on commit/PR; Docker for backend deployment; AWS for hosting services.

3. Project File Structure

Create files in this exact structure. Do not reorganize without a // ARCH DECISION:
comment.

Backend

```
CS4485_Capstone/
  Readme.md
  run.py          # Root entry script (optional wrapper)

  database/
    schema.sql    # SQL schema for persistent storage

  docs/
    API_Contract.md    # API endpoints, request/response contracts
    Architecture.md    # System design and module relationships
    Proposal.md        # Project proposal
    brainstorming.txt  # Early planning notes
    pseudocode.py      # High-level algorithm drafts

  examples/
    fakedoc.md
    example_basic_ast.py    # AST parsing demo
    example_compare.py     # Comparison demo
    example_fingerprinting.py # Fingerprinting demo
    sample_code_v1.py
    sample_code_v2.py

  MeetingMinutes/
    README.md
    CS4485_T2_MOM1.md
    CS4485_T2_MOM2.md
    CS4485_T2_MOM3.md
    WeeklyStatusReport.md

  src/
    __init__.py
    run.py          # Main application runner (src-level)

    config.py       # Config/constants/threshold defaults
    models.py       # Shared domain data models
    persistence.py  # DB/file persistence layer

    ast_parser.py   # Parse source into AST representations
    fingerprint.py  # Generate structural/code fingerprints
    comparator.py   # Similarity comparison logic
    flagging_threshold.py # Rule/threshold-based flag decisions
    alerts.py       # Alert generation/notification logic
    report_generation.py # Human-readable report output

    # Suggested future growth :
    # api/           # API layer (if service endpoints are added)
    # tests/         # Unit/integration tests
    # utils/         # Shared helper functions
```

Frontend

```
CS4485_Capstone_Frontend/  
public/  
  
src/  
  components/  
    issues/      # IssueTable, IssueFilters, IssueDetailPanel  
    reports/     # ReportList, ReportViewer, ExportButton  
    shared/      # Button, Card, Modal, Table, Badge, Spinner  
  
  pages/  
    DashboardPage.tsx  
    IssuesPage.tsx  
    ReportsPage.tsx  
    SettingsPage.tsx  
  
  api/  
    client.ts  
    issues.ts  
    reports.ts  
  
  types/  
    issue.ts  
    report.ts  
    api.ts  
  
  hooks/  
    useIssues.ts  
    useReports.ts  
  
  utils/  
    date.ts  
    format.ts  
  
  App.tsx  
  main.tsx  
  
.env.example      # VITE_API_BASE_URL=http://localhost:8000  
package.json  
vite.config.ts  
tsconfig.json
```

4. Data Models

All models are implemented as PostgreSQL tables.

4.1 Projects

```
CREATE TABLE Projects (  
  id UUID PRIMARY KEY,  
  name TEXT NOT NULL,  
  repo_destination TEXT NOT NULL,  
  primary_language TEXT,  
  documentation_format TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4.2 Scans

```
CREATE TABLE Scans (  
  id UUID PRIMARY KEY,  
  project_id UUID NOT NULL,  
  commit_sha TEXT,  
  status TEXT CHECK (status IN ('queued','running','completed','failed')),  
  mismatch_count INTEGER DEFAULT 0,  
  rot_score FLOAT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY (project_id) REFERENCES Projects(id)  
);
```

4.3 CodeElements

```
CREATE TABLE CodeElements (  
  id UUID PRIMARY KEY,  
  scan_id UUID NOT NULL,  
  element_type TEXT,  
  name TEXT NOT NULL,  
  signature TEXT,  
  file_path TEXT,  
  start_line INTEGER,  
  end_line INTEGER,  
  hash TEXT NOT NULL,  
  visibility TEXT,  
  return_type TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY (scan_id) REFERENCES Scans(id)  
);
```

4.4 DocumentationFiles


```

CREATE TABLE DocumentationFiles (
  id UUID PRIMARY KEY,
  scan_id UUID NOT NULL,
  file_path TEXT NOT NULL,
  file_hash TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (scan_id) REFERENCES Scans(id)
);

```

4.5 DocumentationRefs

```

CREATE TABLE DocumentationRefs (
  id UUID PRIMARY KEY,
  scan_id UUID NOT NULL,
  doc_file_id UUID NOT NULL,
  reference_text TEXT NOT NULL,
  reference_type TEXT,
  line_start INTEGER,
  line_end INTEGER,
  snippet TEXT,
  linked_code_element_id UUID,
  confidence FLOAT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (scan_id) REFERENCES Scans(id),
  FOREIGN KEY (doc_file_id) REFERENCES DocumentationFiles(id),
  FOREIGN KEY (linked_code_element_id) REFERENCES CodeElements(id)
);

```

4.6 Issues

```

CREATE TABLE Issues (
  id UUID PRIMARY KEY,
  scan_id UUID NOT NULL,
  issue_type TEXT,
  severity TEXT CHECK (severity IN ('low','medium','high','critical')),
  status TEXT CHECK (status IN ('open','ignored','resolved')) DEFAULT 'open',
  code_element_id UUID,
  documentation_ref_id UUID,
  documentation_file_path TEXT,
  title TEXT,
  description TEXT,
  detected_hash TEXT,
  expected_hash TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  resolved_at TIMESTAMP,

```

```
FOREIGN KEY (scan_id) REFERENCES Scans(id),
FOREIGN KEY (code_element_id) REFERENCES CodeElements(id),
FOREIGN KEY (documentation_ref_id) REFERENCES DocumentationRefs(id)
);
```

4.7 Reports

```
CREATE TABLE Reports (
  id UUID PRIMARY KEY,
  scan_id UUID NOT NULL,
  format TEXT CHECK (format IN ('txt','json','html','markdown')),
  storage_path TEXT,
  content TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (scan_id) REFERENCES Scans(id)
);
```

4.8 AI Suggestions (Phase 3)

```
CREATE TABLE AISuggestions (
  id UUID PRIMARY KEY,
  issue_id UUID NOT NULL,
  provider TEXT,
  model TEXT,
  suggestion_text TEXT,
  structured_json TEXT,
  confidence FLOAT,
  latency_ms INTEGER,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY (issue_id) REFERENCES Issues(id)
);
```

5. Phase 1 — Planning & Design

Weeks 1–2 · Goal: Finalize architecture, define mismatch rules, and produce test datasets before any code is written.

Phase 1 is complete when: the technical design document is approved, system architecture is defined, mismatch scoring rules are finalized, and at least one test repository with intentional mismatches exists.

Deliverables

- Technical design document
- Architecture diagram
- Defined mismatch rules and scoring weights
- Test repositories with intentional mismatches

Build Order

1. Finalize MVP feature list and language/doc format targets
2. Define system architecture and module boundaries
3. Document mismatch rules and scoring model
4. Create synthetic test repositories

6. Phase 2 — Core Scanner MVP

Weeks 3–5 · Goal: A developer can run the CLI tool against a Python repository and receive a report of detected documentation rot.

Phase 2 is complete when: python run.py scans a repo, detects changes between runs, generates .docrot-report.json and .docrot-report.txt, and core modules reach at least 70% test coverage.

F-201: AST Parsing & Fingerprinting

ID	Requirement	Acceptance Criteria
R-201	Tool parses all .py files in a given directory	ast_parser.py walks the repo and extracts all function/method nodes. Files with syntax errors are skipped with a [docrot] Warning: message.
R-202	Each function produces a deterministic fingerprint	fingerprint.py extracts 7 features (signature, control flow, conditions, calls, side effects, exceptions, returns). Identical source produces identical hash across OS and runs.
R-203	Docstring-only and formatting-only changes produce no events	normalize_function_ast() strips leading docstrings before hashing. Comment-only edits produce zero ChangeEvents.

ID	Requirement	Acceptance Criteria
R-204	Each function has a stable ID	Stable ID is file_path/function_name using forward slashes, used as the dict key in fingerprint storage.

F-202: Baseline Persistence

ID	Requirement	Acceptance Criteria
R-205	Fingerprints saved after every run	persistence.py writes .docrot-fingerprints.json to repo root after each scan.
R-206	First run stores baseline without emitting alerts	is_first_run() returns True when no fingerprint file exists or file is empty. No ChangeEvents emitted; baseline notice printed to stdout.
R-207	Corrupt or missing fingerprint file handled gracefully	Returns empty dict, prints [docrot] Warning: could not read fingerprints; treating as first run. The pipeline continues normally.

F-203: Change Detection & Scoring

ID	Requirement	Acceptance Criteria
R-208	Functions with identical hashes are skipped	Hash fast-path in compare_file_functions() skips feature-level diff when hashes match.
R-209	Modified functions are scored with weighted model	score_semantic_delta() applies the scoring table (0/1/3/5/6/8 points) and returns a SemanticDelta with total score and list of ChangeEvent objects.
R-210	Public function additions and removals are always critical	function_added and function_removed where is_public=True always produce critical=True, score=5 regardless of threshold config.
R-211	Documentation files flagged when thresholds exceeded	evaluate_doc_flags() flags a doc if any mapped function has a critical event OR cumulative score >= per_doc_cumulative threshold.

F-204: CLI Output & Reports

ID	Requirement	Acceptance Criteria
R-212	Stdout follows [docrot] prefix conventions	All status/warning/error lines prefixed with [docrot]. Structured report block printed after all status lines.
R-213	.docrot-report.json written when ChangeEvents exist	Conforms to schema in Section 9.4. Not written on the first run or clean runs. OSError on write is non-fatal.
R-214	.docrot-report.txt written on all non-first runs	Human-readable report. Writes an all-clear message when no flags exist.

ID	Requirement	Acceptance Criteria
R-215	Clean run prints confirmation	When zero ChangeEvents: checkmark No code changes detected since last scan.

Phase 2 — Out of Scope

- Web dashboard or database
- CI/CD integration
- AI suggestions
- Multi-language support

Phase 2 — Build Order

1. models.py — FunctionFingerprint, SemanticDelta, ChangeEvent, DocAlert dataclasses
2. ast_parser.py — AST walking, function extraction, stable ID generation
3. fingerprint.py — 7 feature extractors, normalization, hashing
4. persistence.py — load/save fingerprints, is_first_run()
5. comparator.py — feature diff, scoring engine
6. config.py — config loading, doc mapping, threshold defaults
7. alerts.py — evaluate_doc_flags, publish to log and report
8. report_generation.py — JSON and TXT report writers
9. run.py — CLI entry point wiring all modules
10. Tests for all modules targeting at least 70% coverage

7. Phase 3 — Backend, Database & Web Dashboard

Weeks 6–10 · Goal: Scan results are persisted to a PostgreSQL database, exposed via a FastAPI backend, and viewable in a React dashboard with issue management.

Prerequisite: All Phase 2 requirements must pass before starting Phase 3.

Phase 3 is complete when: the API service is running, scan results are stored in the database, and the dashboard displays projects, scan history, and issues with filtering, search, and status management.

Phase 3 — Database Setup Required

Run database/schema.sql to create all tables: Projects, Scans, CodeElements, DocumentationFiles, DocumentationRefs, Issues, Reports.

F-301: FastAPI Backend

ID	Requirement	Acceptance Criteria
R-301	POST /api/project creates a new project	Accepts name, repo_destination, primary_language. Returns created Project row.
R-302	POST /api/scans triggers a scan	Accepts project_id. Creates a Scan record, runs the scanner, persists results. Status transitions queued > running > completed/failed.
R-303	GET /api/scans returns scan history	Returns scans for a project, newest-first, with status, mismatch_count, rot_score, and commit_sha.
R-304	GET /api/issues returns all issues	Filterable by severity and status. Returns issue title, severity, doc file path, and status.
R-305	GET /api/issues/:id returns issue detail	Returns full issue including code element name, file path, signature, doc snippet, description, and suggestion.
R-306	PATCH /api/issues/:id updates issue status	Accepts status: open ignored resolved. Sets resolved_at when resolved.
R-307	All endpoints validate input with Pydantic	Invalid requests return 422 with field-level error detail.

F-302: Authentication

ID	Requirement	Acceptance Criteria
R-308	Admin login with token-based access	MVP supports a static admin token or simple login. JWT issued for dashboard sessions.
R-309	Protected routes require valid token	All /api/* routes return 401 for missing or invalid tokens.
R-310	Optional GitHub OAuth integration	If configured, users can authenticate via GitHub. On success, upsert user record and issue JWT.

F-303: React Dashboard

ID	Requirement	Acceptance Criteria
R-311	Dashboard shows project list and recent scan summary	DashboardPage.tsx shows each project with the latest rot score and mismatch count.
R-312	Issues page shows filterable, searchable issue table	IssuesPage.tsx supports filtering by severity and status, and keyword search by function name, filename, or general keyword.
R-313	Issue detail panel shows full context	Clicking a row opens a panel with code element name, file path, signature, doc snippet, description, and suggestion if available.

ID	Requirement	Acceptance Criteria
R-314	User can resolve, ignore, or create a ticket for an issue	The Detail panel has Resolve, Ignore, and Create Ticket buttons. Status changes call PATCH /api/issues/:id. Optimistic UI update on change.
R-315	Scan history page shows past scans	ReportsPage.tsx lists scans with timestamp, commit SHA, status, and mismatch count. Clicking a scan shows its report.
R-316	Empty states shown when no data exists	All pages show a descriptive empty state when no projects, scans, or issues exist.

F-304: CI/CD Integration

ID	Requirement	Acceptance Criteria
R-317	GitHub Actions workflow triggers on push and PR	.github/workflows/docrot.yml runs the scanner on every push to main and every pull request.
R-318	Scanner posts results to backend API	After scanning, the Action calls the backend to create a Scan record and persist issues. Requires DOCROT_API_URL and DOCROT_API_TOKEN secrets.
R-319	Action exits with code 1 on critical issues	CI pipeline fails when any critical=True ChangeEvent is detected.
R-320	Commit SHA captured and stored	--commit argument passed to scanner, stored in Scans.commit_sha.

F-305: AI-Assisted Suggestions

ID	Requirement	Acceptance Criteria
R-321	Suggestions generated for high/critical issues post-scan	Backend calls OpenAI or Gemini for each Issue with severity high or critical. Results stored in AISuggestions.
R-322	Prompt includes full issue context	Prompt includes function name, signature, file path, change reason, and doc snippet if available.
R-323	Suggestion displayed in issue detail panel	Suggestion text shown below issue description with a subtle AI label. Not shown if none exists.
R-324	AI failure is non-fatal	If AI is unavailable, the issue is saved without a suggestion. Scan status remains completed.
R-325	System degrades gracefully without AI configured	If no API key is set, suggestion generation is skipped silently. No errors surfaced to users.

Phase 3 — Build Order

1. Run database schema migrations
2. FastAPI app setup — project and scan endpoints
3. Issues and reports endpoints
4. Token-based auth middleware
5. GitHub Actions workflow

6. React dashboard scaffolding — DashboardPage, IssuesPage, ReportsPage
7. Issue detail panel with resolve/ignore/create ticket actions
8. Search and filter functionality
9. AI suggestion generation and display
10. Tests for all API endpoints and AI routes

8. API References

Base URL: /api/v1. All responses are JSON. Dashboard-protected routes require Authorization: Bearer <accessToken> or valid admin token, depending on MVP auth mode. Timestamps are ISO 8601 UTC.

API Adjustment Note

The original FastAPI-based REST API design has been partially replaced for the MVP. Instead of all scan results being processed through REST endpoints, GitHub Actions now send scan data directly to a Cloud Function ([ingestScan](#)), which persists results into Firestore.

Some REST-style endpoints remain conceptually relevant for future expansion but are not fully implemented in the current MVP.

8.1 Auth Routes

POST /auth/login

Request body { email?: string, password?: string, adminToken?: string }

Success 200 { user: { id, email, role }, accessToken: string }

Mode MVP supports either simple admin login or static admin token access.

Error 401 Invalid credentials or invalid admin token. Generic message only.

Error 422 Validation failure — field-level errors in details.

POST /auth/refresh

Request No body. Reads refresh/session token if JWT session mode is enabled.

Success 200 { accessToken: string }

Error 401 Missing, expired, or invalid token. Client must re-login.

Note Optional for MVP. May be omitted if project uses static admin token mode only.

POST /auth/logout

Auth Protected

Success 200 { message: "Logged out" }

Side effects Invalidates server session or refresh token if JWT mode is enabled.

GET /auth/me

Auth Protected

Success 200 { user: { id, email, role } }

Error 401 Missing or invalid authentication.

GET /auth/github + GET /auth/github/callback

Flow Optional GitHub OAuth flow.

On success Upsert user record and issue JWT/session for dashboard access.
On failure Return 401 or redirect to login error page, depending on frontend auth flow.
Note Optional integration only. Not required for MVP.

8.2 Project Routes (all protected)

GET /projects

Success 200 { projects: Project[] }

Returns All registered projects in the system.

POST /projects

Body { name: string, repo_destination: string, primary_language: string, documentation_format?: string }

Success 201 { project: Project }

Constraint name is required. repo_destination is required. primary_language is required.
documentation_format is optional but should default to the configured MVP format.

Error 409 Project with same repo_destination already exists.

Error 422 Validation failure — field-level errors in details.

GET /projects/:id

Success 200 { project: Project }

Error 404 Project not found.

PATCH /projects/:id

Body { name?: string, repo_destination?: string, primary_language?: string, documentation_format?: string }

Success 200 { project: Project }

Constraint Empty strings not allowed for required fields.

Error 404 Project not found.

DELETE /projects/:id

Success 200 { message: "Project deleted" }

Side effects Deletes linked scans, code elements, documentation files, documentation refs, issues, and reports for that project unless soft-delete is implemented later.

Error 404 Project not found.

8.3 Scan Routes (all protected)

POST /scans

Body { project_id: string, commit_sha?: string }

Success 201 { scan: Scan }

Side effects Creates a Scan record, transitions status queued > running > completed or failed, runs the scanner, and persists results to the database.

Constraint project_id must reference an existing project.

Error 404 Project not found.

Error 409 Returns conflict if a scan is already running for the same project and concurrent scans are not allowed.

GET /scans

Query params project_id?: string, page: number (default 1), limit: number (default 20, max 100)

Success 200 { scans: Scan[], total: number, page: number, totalPages: number }

Default Sort newest first by created_at.

Returns Scans across all projects if project_id is omitted, or only the specified project if provided.

GET /scans/:id

Success 200 { scan: Scan }

Includes id, project_id, commit_sha, status, mismatch_count, rot_score, created_at, updated_at.

Error 404 Scan not found.

GET /projects/:id/scans

Query params page: number (default 1), limit: number (default 20, max 100)

Success 200 { scans: Scan[], total: number, page: number, totalPages: number }

Returns Scan history for one project only, newest first.

Error 404 Project not found.

GET /scans/:id/report

Success 200 { report: Report }

Returns The primary stored report for the scan.

Error 404 Scan not found.

Error 404 Report not found for the scan.

8.4 Issue Routes (all protected)

GET /issues

Query params severity?: "low" | "medium" | "high" | "critical", status?: "open" | "ignored" | "resolved", search?: string, project_id?: string, scan_id?: string, page: number (default 1), limit: number (default 20, max 100)

Success 200 { issues: Issue[], total: number, page: number, totalPages: number }

Filterable By severity and status.

Search Supports keyword search by function name, filename, or general keyword.

Returns Issue title, severity, documentation_file_path, status, and related summary fields for list display.

GET /issues/:id

Success 200 { issue: IssueDetail }

Returns Full issue including code element name, file path, signature, doc snippet, description, and suggestion if available.

Error 404 Issue not found.

GET /scans/:id/issues

Query params severity?: "low" | "medium" | "high" | "critical", status?: "open" | "ignored" | "resolved", page: number (default 1), limit: number (default 20, max 100)

Success 200 { issues: Issue[], total: number, page: number, totalPages: number }

Returns All issues generated from the specified scan.

Error 404 Scan not found.

PATCH /issues/:id

Body { status: "open" | "ignored" | "resolved" }

Success 200 { issue: Issue }

Constraint status must be one of open, ignored, or resolved.
Side effects Sets resolved_at when status changes to resolved. Clears resolved_at when issue is reopened to open.
Error 404 Issue not found.
Error 422 Validation failure — field-level errors in details.

POST /issues/:id/create-ticket

Body { provider?: string }

Success 201 { message: "Ticket created", external_id?: string }

Note Optional integration route used by the dashboard Create Ticket button.

Constraint If external ticketing integration is not enabled, return 501 Not Implemented.

Error 404 Issue not found.

8.5 Report Routes (all protected)

GET /reports

Query params project_id?: string, scan_id?: string, format?: "txt" | "json" | "html" | "markdown", page: number (default 1), limit: number (default 20, max 100)

Success 200 { reports: Report[], total: number, page: number, totalPages: number }

Returns Stored reports across all projects or filtered by project/scan.

GET /reports/:id

Success 200 { report: Report }

Includes id, scan_id, format, storage_path, content, created_at.

Error 404 Report not found.

POST /reports

Body { scan_id: string, format: "txt" | "json" | "html" | "markdown" }

Success 201 { report: Report }

Side effects Generates and stores the requested report format for the scan if it does not already exist.

Constraint scan_id must reference an existing scan.

Error 404 Scan not found.

Error 409 Scan has not completed yet.

8.6 Documentation Reference Routes (all protected)

GET /documentation-files/:id

Success 200 { documentationFile: DocumentationFile }

Returns Metadata for one documentation file analyzed during a scan.

Error 404 Documentation file not found.

GET /documentation-files/:id/refs

Success 200 { documentationRefs: DocumentationRef[] }

Returns All extracted documentation references for the specified documentation file.

Error 404 Documentation file not found.

GET /documentation-refs/:id

Success 200 { documentationRef: DocumentationRef }

Includes reference_text, reference_type, line_start, line_end, snippet, linked_code_element_id,

confidence.

Error 404 Documentation reference not found.

GET /code-elements/:id

Success 200 { codeElement: CodeElement }

Includes element_type, name, signature, file_path, start_line, end_line, hash, visibility, return_type.

Error 404 Code element not found.

8.7 Dashboard Summary Routes (all protected)

GET /dashboard/summary

Query params project_id?: string

Success 200 { projects: number, scans: number, openIssues: number, latestScan?: { id, created_at, mismatch_count, rot_score, status } }

Note Used by DashboardPage.tsx to show project list and recent scan summary cards.

GET /dashboard/projects

Success 200 { projects: [{ id, name, latest_rot_score, latest_mismatch_count, latest_scan_status, latest_scan_at }] }

Note Used for the dashboard project list view.

8.8 Search & Filter Behavior

GET /issues with search/filter query params

Behavior severity and status filters are exact-match filters.

Behavior search performs case-insensitive matching against code element name, documentation_file_path, signature, and issue title/description when available.

Behavior empty filter params are ignored.

Behavior results are sorted newest first unless otherwise specified.

8.9 AI Routes — Phase 3 Optional / AI-Assisted Suggestions

POST /ai/issues/:id/suggest

Auth Protected

Success 200 { suggestion: AISuggestion }

Logic Generates a suggestion for a high or critical issue using OpenAI when AI is configured.

Prompt includes function name, signature, file path, change reason, and doc snippet if available.

Error 404 Issue not found.

Error 501 AI is not configured or feature is disabled.

Note AI failure is non-fatal to the overall scan pipeline.

GET /ai/issues/:id/suggestions

Auth Protected

Success 200 { suggestions: AISuggestion[] }

Returns All stored suggestions for the issue, newest first.

Error 404 Issue not found.

8.10 CI / Automation Routes

POST /automation/scan-results

Auth Protected via DOCROT_API_TOKEN or equivalent service token.

Body { project_id: string, commit_sha?: string, scan_payload: object }

Success 201 { scan: Scan, persistedIssues: number }

Use case GitHub Actions posts results to backend after running the scanner.

Constraint Used by CI pipeline integration in Phase 3.

Error 401 Missing or invalid API token.

Error 404 Project not found.

POST /automation/trigger-scan

Auth Protected via DOCROT_API_TOKEN or equivalent service token.

Body { project_id: string, commit_sha?: string }

Success 202 { message: "Scan queued", scan_id: string }

Use case Internal or CI route to trigger a backend-managed scan for a project.

Error 401 Missing or invalid API token.

Error 404 Project not found.

8.11 Health Routes

GET /health

Success 200 { status: "ok" }

Note Lightweight liveness endpoint for backend monitoring.

GET /health/ready

Success 200 { status: "ready", database: "ok" }

Error 503 Returned when required services are unavailable.

8.12 Response Object Reference

Project

{ id, name, repo_destination, primary_language, documentation_format, created_at }

Scan

{ id, project_id, commit_sha, status, mismatch_count, rot_score, created_at, updated_at }

CodeElement

{ id, scan_id, element_type, name, signature, file_path, start_line, end_line, hash, visibility, return_type, created_at }

DocumentationFile

{ id, scan_id, file_path, file_hash, created_at }

DocumentationRef

{ id, scan_id, doc_file_id, reference_text, reference_type, line_start, line_end, snippet, linked_code_element_id, confidence, created_at }

Issue

{ id, scan_id, issue_type, severity, status, code_element_id, documentation_ref_id, documentation_file_path, title, description, detected_hash, expected_hash, created_at, updated_at, resolved_at }

IssueDetail

```
{ id, scan_id, issue_type, severity, status, title, description, detected_hash, expected_hash, created_at, updated_at, resolved_at, codeElement: CodeElement | null, documentationRef: DocumentationRef | null, suggestion: AISuggestion | null }
```

Report

```
{ id, scan_id, format, storage_path, content, created_at }
```

AISuggestion

```
{ id, issue_id, provider, model, suggestion_text, structured_json, confidence, latency_ms, created_at }
```

8.13 Common Error Responses

Error 400 Bad request — malformed JSON or invalid query parameters.

Error 401 Unauthorized — missing or invalid authentication token.

Error 404 Not found — requested resource does not exist.

Error 409 Conflict — invalid state transition or conflicting scan already in progress.

Error 422 Validation failure — field-level validation errors returned in details.

Error 500 Internal server error — unexpected backend failure.

Error 501 Not implemented — optional integration or AI feature not enabled.

9. CLI & Output Guidelines

All user-facing output from the Documentation Rot Detector — whether written to stdout, to a text report, or to a JSON artifact — must follow the conventions in this section. Consistency ensures the tool integrates cleanly into CI pipelines, developer terminals, and post-MVP tooling without requiring interpretation.

9.1 Stdout Conventions

Every line written to stdout must be prefixed with [docrot] when it is a status or warning message emitted by the tool itself, as opposed to structured report output. This allows CI log parsers and shell scripts to grep for tool messages reliably.

Prefix / Style	When Used	Example
[docrot]	Status messages, progress updates, warnings	[docrot] Found 42 Python file(s). Extracting fingerprints...

[docrot] Warning:	Recoverable issues that do not stop the pipeline	[docrot] Warning: could not read src/legacy.py: PermissionError
[docrot] Error:	Fatal errors that cause the tool to exit with code 2	[docrot] Error: 'not-a-dir' is not a valid directory.
No prefix	Structured report sections (separator lines, change tables, severity summary)	

The structured stdout report block is always printed after all [docrot] status lines. It is never interleaved with them. The block is omitted entirely on first run — only the baseline notice is printed.

9.2 Stdout Report Block Structure

On every subsequent run, the following sections appear in this fixed order, separated by a line of — characters (64 wide):

Section	Always Printed?	Content
Header	Yes	Files scanned, functions found, scan time in seconds
CODE CHANGES	Only if ChangeEvents exist	Critical events first (Δ), then non-critical (i). Each entry shows event type, stable_id, reasons, and score.
DOCUMENTATION FILES FLAGGED	Only if ChangeEvents exist	One block per DocAlert. Shows CRITICAL or WARNING tag, doc path, cumulative score, reasons, and affected function IDs.

SEVERITY SUMMARY	Only if ChangeEvents exist	Single line: High count, Medium count, Low count across all Flag objects.
Reports written	Only if reports were generated	Paths to .docrot-report.txt and .docrot-report.json
Clean confirmation	Only if zero ChangeEvents	Single line: ✓ No code changes detected since last scan.

9.3 Severity Display Labels

Severity levels must be rendered consistently across all output surfaces. The canonical labels and their usage context are:

Severity	Stdout Tag	TXT Report Tag	JSON Value	Color (future UI)
HIGH	⚠ CRITICAL	[HIGH]	"high"	#EF4444 (red)
MEDIUM	⚠ WARNING	[MEDIUM]	"medium"	#F59E0B (amber)
LOW	ℹ NOTICE	[LOW]	"low"	#6B7280 (gray)

Severity is never conveyed by visual style alone. Every severity indicator in stdout and report output is paired with its text label. Color is reserved for post-MVP UI surfaces and must never be the sole means of conveying state.

9.4 JSON Report Schema

The `.docrot-report.json` file written by `report_generation.generate_json_report()` must conform to the following top-level structure on every run that produces at least one `ChangeEvent`:

```
{
  "meta": {
    "repo_path":      string,    // absolute path to scanned repo root
    "commit_hash":    string,    // value of --commit arg, or "unknown"
    "timestamp":      string,    // ISO 8601 datetime of scan start
    "total_issues":   number,    // total Flag count
    "severity_summary": { "high": number, "medium": number, "low": number }
  },
  "issues": [ <Flag>, ... ]    // sorted HIGH → MEDIUM → LOW
}
```

Each issue object within the issues array must include: reason (string enum value), severity (string enum value), code_element (name, file_path, signature, params, return_type), doc_reference (nullable object with file_path, referenced_symbol, snippet), message, and suggestion.

The file is never written on a clean run (zero `ChangeEvents`). It is never written on a first run. If the file cannot be written due to an `OSError`, the error is printed to stdout with the `[docrot] Error:` prefix and the tool continues to exit normally — report write failure is non-fatal.

9.5 TXT Report Schema

The `.docrot-report.txt` file is a plain-text, human-readable equivalent of the JSON report. It must be writable without a terminal and must render correctly in any monospace viewer, CI log viewer, or email attachment.

Section	Format
Header block	Two lines of = characters (60 wide) surrounding repo path, commit hash, timestamp, and total issue count with severity breakdown
Issue block (one per Flag)	Line of - characters (60 wide), then labeled fields: Reason, Symbol, Code file, Signature (if non-empty), Doc file (if present), Doc snippet (first 80 chars, if present), blank line, ! message, > Suggestion

All-clear block	Printed instead of issue blocks when no flags exist: 'No documentation rot detected. All docs are up to date!'
Footer block	Two lines of = characters with total issue count

9.6 Suggestion Text Standards

Every Flag object must carry a non-empty suggestion string. Suggestions are surfaced in both the TXT report and, post-MVP, in CI annotations. They must follow these rules:

- **Actionable:** the suggestion must describe a specific action, not restate the problem. 'Review documentation' is acceptable. 'Something changed' is not.
- **Named:** the suggestion must reference the affected function by its stable_id or short name so the developer knows exactly where to look.
- **Terse:** suggestions must fit on a single line. No multi-sentence suggestions.
- **No stack traces:** internal error details must never appear in suggestion text.

10. Error Handling Standards

The Documentation Rot Detector operates in developer terminals and unattended CI environments. Error handling must therefore be conservative: prefer graceful degradation over crashes, always log context with [docrot] prefixes, and never silently swallow errors that could produce incorrect results.

10.1 Error Classification

Class	Behavior	Exit Code	Examples
Fatal	Print [docrot] Error: message, exit immediately	2	repo_path does not exist or is not a directory

Recoverable	Print [docrot] Warning: message, skip affected item, continue pipeline	0 or 1 (normal)	Unreadable source file, corrupt config, corrupt fingerprint file
Silent skip	No output, item excluded from results	0 or 1 (normal)	Source file with syntax errors (parse_source returns None); empty files with no functions
Non-fatal write failure	Print [docrot] Error: could not write..., continue to exit normally	0 or 1 (normal)	OSError writing .docrot-report.json or .docrot-fingerprints.json

10.2 Per-Module Error Handling Rules

config.py

- **Missing config file:** fall back to DEFAULT_CONFIG silently. No warning — absence is expected for new repos.
- **Malformed JSON or OSError:** print [docrot] Warning: could not read config (<error>); using defaults. Fall back to DEFAULT_CONFIG.
- **Missing keys in config:** use DEFAULT_CONFIG values for any absent key. Never raise KeyError.

persistence.py

- **Missing fingerprint file:** return empty dict. This is the normal first-run state — no warning printed.
- **Corrupt / unreadable fingerprint file:** print [docrot] Warning: could not read fingerprints (<error>); treating as first run. Return empty dict.
- **Non-dict top-level value:** return empty dict. Same behavior as corrupt file.
- **OSError on write:** print [docrot] Error: could not write fingerprints: <error>. Do not raise. The pipeline continues and exits with its normal code.
- **Failed file reads during scan:** the failed file's old fingerprint must NOT be replaced in the persisted baseline. Only fingerprints successfully extracted on the current run may overwrite stored values.

ast_parser.py / fingerprint.py

- **SyntaxError during parse:** print [docrot] Syntax error while parsing source: <error>. Return None from parse_source(). The caller (_scan_repo) skips the file.

- **Any file yielding zero function nodes:** excluded from current_fps silently. Not an error.
- **Unexpected AST node types:** unknown nodes are skipped inside ast.walk loops. Never raise on unrecognized node types.

comparator.py

- **Function in old but not new:** treated as function_removed. Only emits a critical event if the old fingerprint marks is_public=True.
- **Function in new but not old:** treated as function_added. Only emits a critical event if is_public=True.
- **Identical hashes:** skipped entirely. No event, no diff, no output.

report_generation.py

- **OSError writing .docrot-report.json or .docrot-report.txt:** the exception propagates to run.py. run.py must catch it, print [docrot] Error: could not write report: <error>, and continue to the stdout summary and fingerprint persistence steps.
- **Empty flags list:** generate_reports() must still write both files. The TXT report writes the all-clear message. The JSON report writes an issues array of length zero

10.3 What Must Never Happen

- An unhandled exception must never reach the user's terminal. The top-level run() call in __main__ may wrap the pipeline in a broad try/except as a last-resort guard, printing a [docrot] Internal error: message and exiting with code 2.
- A stack trace must never appear in a report file (.json or .txt). Internal error details belong only on stderr or in a future debug log.
- A failed fingerprint write must never silently leave the baseline in a partially-written or truncated state. Use atomic write patterns (write to a temp file, rename) in post-MVP iterations; for MVP, log the error and accept the risk.
- A recoverable warning must never cause the tool to exit with code 2. Code 2 is reserved exclusively for the fatal error class defined in section 10.1.
- The tool must never print a Python traceback to stdout in production use. Tracebacks are acceptable in development with --debug (post-MVP).

10.4 First Run Edge Cases

The following edge cases during first-run baseline creation must be handled explicitly:

Scenario	Required Behavior
----------	-------------------

Repo has zero .py files	Print '[docrot] No Python files found. Exiting.' and exit with code 0. Do not write a fingerprint file.
All .py files fail to parse	current_fps is empty. Persist an empty fingerprint file. Print baseline notice. Exit 0. On next run this will appear as a first run again.
Fingerprint file exists but is empty ({})	is_first_run() returns True. Treat as first run. Overwrite with current fingerprints.
Config file missing on first run	Use DEFAULT_CONFIG. Print no warning. doc_mappings will be empty, which is acceptable — the user is warned about this separately at the start of every run that has no doc_mappings.

11. Non-Functional Requirements

11.1 Correctness

Correctness is the most critical non-functional property of this tool. A false positive (flagging documentation that does not need review) erodes developer trust. A false negative (missing a real semantic change) defeats the tool's purpose entirely.

Requirement	Rationale
Fingerprint hashes must be fully deterministic. Identical source code scanned twice — in any order, on any OS, across runs — must produce the exact same hash.	Non-determinism would produce spurious ChangeEvents on every run, making the tool unusable.
Docstring-only edits must produce zero ChangeEvents. The normalize_function_ast() step must strip the	Developers routinely improve docstrings. Flagging doc changes as doc rot would be self-contradictory.

leading docstring before any feature is extracted or hashed.	
Comment-only and formatting-only edits (whitespace, blank lines) must produce zero ChangeEvents.	These are not semantic changes. The AST representation is already whitespace-independent; no additional normalization is required.
A function whose hash is unchanged must never reach the feature-level diff path. The hash fast-path in <code>compare_file_functions()</code> is mandatory, not optional.	Performance and correctness both depend on skipping unchanged functions.
Failed file reads during a scan must not remove or overwrite the stored baseline fingerprint for that file.	An I/O error mid-scan must not silently make every function in that file appear 'removed', triggering false critical alerts.
Public function additions and removals must always be scored as critical (<code>score=5</code> , <code>critical=True</code>), regardless of threshold configuration.	Adding or removing a public API is always a documentation-relevant event.

11.2 Performance

Target	Threshold	Notes
Scan time — small repo (≤ 500 functions)	< 3 seconds	Measured on commodity developer hardware (mid-range laptop, SSD)
Scan time — medium repo ($\leq 10,000$ functions)	< 30 seconds	Linear scaling expected; dominated by file I/O and AST parsing
Memory — medium repo	< 512 MB RSS	FunctionFingerprint objects are lightweight; full repo held in memory during comparison

Hash comparison fast-path	Must be exercised first	Feature-level diff (diff_features) is only called when fingerprint_hash values differ
File reads	Each file read exactly once per run	No re-reading of source files across pipeline stages; _scan_repo is the single read pass
Fingerprint JSON write	< 1 second for $\leq 10,000$ functions	json.dump with indent=2 on a dict of serialized dataclasses

Performance regressions that push small-repo scan time above 5 seconds or medium-repo time above 60 seconds must be treated as bugs and resolved before merge. Profiling should target ast.walk calls and json.dump serialization as the most likely hotspots.

11.3 Robustness & Reliability

- **Crash-free operation:** the tool must not raise an unhandled exception under any combination of valid inputs, including repos with unusual file layouts, files with encoding edge cases (non-UTF-8 bytes are skipped via errors='replace' in post-MVP; MVP uses UTF-8 strict), or extremely large functions (10,000+ AST nodes).
- **Partial repo coverage:** if N% of files fail to parse, the tool must still produce a valid report for the remaining (100-N)% of the repo. Partial coverage is preferable to a full crash.
- **Idempotent baseline writes:** running the tool twice on an unchanged repo must produce identical fingerprint files (same JSON content, same keys, same hashes). The second run must emit no ChangeEvents.
- **No side effects outside repo root:** the tool must not write any files outside of repo_path. All output artifacts (.docrot-report.json, .docrot-report.txt, .docrot-fingerprints.json) are written to repo_path only.
- **Concurrent run safety (MVP):** no file locking is implemented. Running two instances of the tool simultaneously against the same repo_path may produce corrupt fingerprint files. This is a known MVP limitation. Post-MVP must implement advisory file locking.

11.4 Code Quality

Standard	Rule
Type annotations	All public functions and methods must have complete type annotations on parameters and return values. No implicit Any.
Docstrings	All public functions must have a docstring describing purpose, args, and return value. Private helpers (_prefixed) are encouraged but not required.
Import discipline	No circular imports. run.py may import from all src/ modules. src/ modules must not import from run.py or from each other in a circular fashion. report_generation.py and flagging_threshold.py are treated as peer modules (not under src/).
No global mutable state	All pipeline state is passed explicitly as function arguments. No module-level mutable variables that change between runs.
Test coverage (MVP target)	≥ 70% line coverage across ast_parser.py, fingerprint.py, comparator.py, alerts.py, and persistence.py. run.py and report_generation.py may be tested via integration tests.
Linting	Code must pass flake8 with max-line-length=100 and no E501 suppressions on logic lines. Blank-line suppressions (E302, E303) are permitted in dataclass files.
No print() in library modules	Only run.py and alerts.py may call print() directly. All other modules must return values or raise exceptions — never print to stdout themselves.

11.5 Platform Compatibility

- **Python version:** Python 3.10 minimum. Tested on Python 3.14. No walrus operators (:=) in hot paths for 3.7 back-compat (dropped — 3.10 is the floor).

- **Operating systems:** Windows (PowerShell, py launcher), macOS, Linux. All three must be supported without OS-specific code branches.
- **Path separators:** all internal path keys and stable_id strings must use forward slashes (/), regardless of host OS. os.path.relpath() output must be normalized with .replace('\\', '/') before use as any dict key, stable ID component, or report field.
- **Encoding:** all file reads and writes must specify encoding='utf-8' explicitly. Relying on the platform default locale encoding is not permitted.
- **No external runtime dependencies (MVP):** the tool must run with Python's standard library only. The only permitted third-party import is in report_generation.py's use of datetime, which is stdlib. Post-MVP may introduce optional dependencies (e.g. rich for colored output) behind a try/except import guard.

11.6 Security

- **No code execution:** the tool must never execute, import, or eval any user source code. All analysis is performed solely through Python's ast module on source strings. This is a hard requirement — dynamic analysis is out of scope for all phases.
- **No network access:** the tool must never make network requests. It operates entirely on local filesystem state.
- **Path traversal:** repo_path is resolved to an absolute path via os.path.abspath() before use. All relative paths produced by os.walk are validated to remain within repo_path. The tool must not follow symlinks that escape the repo root (post-MVP: add os.path.realpath check on each discovered file).
- **Fingerprint file integrity:** the .docrot-fingerprints.json file is read-only trusted input. If its top-level value is not a dict, it is discarded and the run proceeds as a first run. No attempt is made to execute or interpret values in the fingerprint file beyond JSON deserialization.
- **No credentials or secrets in output:** report files must never contain environment variable values, file contents, or any data beyond the structured fields defined in section 9.4. Source code snippets are not included in any output — only AST-derived feature counts and hashes.

12. Environment Variables

Create .env.example files with all keys present and no values. Backend config must be validated at startup and fail fast if any required variable is missing or invalid. Frontend config must be validated at app startup/build time before API calls are made.

12.1 Backend (.env)

```
DATABASE_URL=
API_HOST=
API_PORT=
NODE_ENV=
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
JWT_ACCESS_EXPIRES_MINUTES=
JWT_REFRESH_EXPIRES_DAYS=
ADMIN_EMAIL=
ADMIN_PASSWORD=
ADMIN_TOKEN=
FRONTEND_URL=
CORS_ALLOWED_ORIGINS=
DOCROT_REPO_BASE_PATH=
DOCROT_REPORT_STORAGE_PATH=
DOCROT_DEFAULT_LANGUAGE=
DOCROT_DEFAULT_DOC_FORMAT=
DOCROT_API_TOKEN=
OPENAI_API_KEY=
OPENAI_MODEL=
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=
GITHUB_CALLBACK_URL=
GITHUB_WEBHOOK_SECRET=
AWS_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_S3_BUCKET=
LOG_LEVEL=
```

12.2 Frontend (.env)

VITE_BACKEND_ORIGIN
VITE_API_BASE_URL
VITE_DOCROT_TOKEN
VITE_SCAN_EVENTS_PATH
VITE_APP_ENV
VITE_ENABLE_AI
VITE_ENABLE_GITHUB_AUTH

13. Glossary

These terms have precise meanings in this codebase. Use them consistently in code, variable names, and comments.

Term	Definition
Documentation Rot	A state where documentation references code behavior, signatures, or features that no longer match the current source code.
Scan	A single execution of the DocRot analysis process against a repository, producing detected issues and reports.
Project	A repository registered in the system for documentation rot monitoring. Stored in the Projects table.
Code Element	A parsed program structure extracted from AST analysis, such as a function, method, or class. Stored in the CodeElements table.
Fingerprint	A deterministic hash generated from structural code features (signature, control flow, conditions, calls, side effects, exceptions, returns). Used to detect semantic changes.
Stable ID	Unique identifier for a function or code element generated as file_path/function_name using forward slashes. Used as a key across scans.
ChangeEvent	A detected change between two scans indicating that a function's fingerprint has changed, been added, or been removed.
Documentation Reference	A detected reference in documentation that points to a code element (e.g., function_name(), class name, API endpoint). Stored in DocumentationRefs.

Issue	A detected documentation mismatch produced when a code change affects referenced documentation. Stored in the Issues table.
Critical Issue	A high-severity issue triggered by significant API changes such as public function addition, removal, or major semantic modification.
Rot Score	A numeric measure of cumulative documentation mismatch severity produced during a scan. Stored in Scans.rot_score.
Baseline	The stored fingerprint state of a repository from the previous scan used to compare against the current run.
Report	A structured output artifact describing detected documentation rot, generated in .docrot-report.json or .docrot-report.txt.
AI Suggestion	An optional AI-generated recommendation describing how documentation should be updated to resolve an issue. Stored in AISuggestions.
CLI Scanner	The command-line tool (run.py) that scans a repository, performs AST analysis, compares fingerprints, and generates reports.
Dashboard	The React web interface used to view projects, scans, issues, and reports from the backend API.
CI Scan	A scan triggered automatically by a CI/CD pipeline (e.g., GitHub Actions) when code changes are pushed or pull requests are opened.
First Run	The initial scan of a repository where no baseline fingerprints exist. Fingerprints are stored but no issues are emitted.
Clean Run	A scan where no ChangeEvents or documentation mismatches are detected compared to the previous baseline.
Suggestion	A single-line actionable recommendation attached to a detected issue explaining how documentation should be updated.

14. Agent Instructions & Constraints

Read this section before writing any code. These rules override any other instinct or default behavior.

14.1 Phase Gates

This specification is divided into three implementation phases. Each phase has a strict gating rule.

- Do NOT start Phase 2 until all Phase 1 deliverables are complete, including the technical design document, architecture diagram, mismatch rules, and synthetic test repositories.
- Do NOT start Phase 3 until all Phase 2 requirements pass, including CLI scanning, baseline persistence, change detection, report generation, and minimum 70% test coverage for core modules.
- Phase 3 development may only begin after Phase 2 produces valid `.docrot-report.json` and `.docrot-report.txt` outputs and the scanner correctly detects documentation rot in the test repositories.
- When beginning Phase 3, run the database schema defined in `database/schema.sql` before implementing API routes or dashboard features.

14.2 Build Order Within Each Phase

- Always build backend logic before frontend functionality.
- Within Phase 2, build modules in the following order:
`models.py` → `ast_parser.py` → `fingerprint.py` → `persistence.py` → `comparator.py` → `config.py` → `alerts.py` → `report_generation.py` → `run.py`.
- Within Phase 3, build components in this order:
Database schema → FastAPI backend → API validation and tests → CI integration → React dashboard pages → frontend integration tests.
- Complete each feature fully before moving to the next requirement group.
- Verify that every requirement ID listed in the specification for that feature passes before proceeding to the next feature.

14.3 Decision Rules

- When the specification is ambiguous, implement the simplest interpretation and add:
`// SPEC QUESTION: [description of ambiguity]`
- When making an architectural decision not explicitly defined in this document, add:
`// ARCH DECISION: [explanation]`
- The specification is the source of truth. If a requirement appears incorrect or inconsistent, implement it exactly as written and add:
`// SPEC ERROR: [concern]`
- Do not add external libraries or frameworks that are not listed in Section 2. If a new dependency is unavoidable, add a `// ARCH DECISION: comment explaining the reason.`
- Do not implement any feature listed as Out of Scope for the current phase.

14.4 Security Rules (Non-Negotiable)

- Never hardcode secrets, tokens, credentials, or environment-specific values in the source code. All configuration must come from environment variables.
- The scanner must never execute, import, or evaluate user code. All analysis must occur through static parsing using Python's `ast` module.
- The backend must validate all incoming API requests using Pydantic models before processing.
- Authentication tokens must be validated for all protected routes. Any `/api/*` route requiring authentication must return HTTP 401 if the token is missing or invalid.
- Secrets such as `OPENAI_API_KEY`, `JWT_*`, `ADMIN_TOKEN`, and database credentials must never appear in frontend code, logs, reports, or API responses.
- Stack traces must never be returned in API responses. Production error responses must contain generic messages only.

14.5 Handling Gaps

If this specification does not cover a situation encountered during implementation, apply these rules in order:

1. Check whether a similar pattern exists elsewhere in the specification and follow it.
2. Choose the implementation that is most consistent with the existing architecture and module boundaries.
3. Prefer deterministic, simple implementations over complex abstractions.
4. Leave a comment explaining the decision using `// ARCH DECISION:`.